

Who Can Turn Off Your Ethereum App?

A Full-Stack Walk-Away Test

James Carnley
Ethereum File System

The archive is still theirs

A community spends years building an archive.

Someone returns for a file.
The builder is gone.

Can they still find it, open it and trust it?

Absence and hostile control

The maintainer disappears

Can users continue without the team's services, accounts or help?

The maintainer's keys are hostile

Can users refuse a change and continue with what they trusted?

Ethereum File System

- An open-source public good for file names and onchain records
- Paths, provenance and verifiable file references
- File bytes can live on different storage networks
- Sepolia is the working example in this talk

The interface and the protocol

Uniswap Labs interface

Labs documents restrictions through its interfaces.

The usual entrance has its own controls.

Underlying protocol

A different access layer.

A user still needs a compatible client and a safe way to use it.

The code the user actually runs

- Retain the runnable release and its dependencies
- Inspect scripts loaded after the app starts
- Test an old release on a fresh device
- Rebuilding from source is a separate check

ENS names point to content.
A browser gateway provides an access route.

Retain the accepted release reference.
Test another route.

Who chooses the next reference?

A familiar name can change

Finding and checking the records

- Storage needs retained copies
- RPC responses need a known chain and block basis
- Indexes need definitions and sufficient history
- A correct result may still be stale or incomplete

Recovery needs the right keys

Public file retrieval

Bytes and a trusted reference may be enough to read.

Our EFS task follows this path.

Private work or new writes

Decryption, signing and account recovery add dependencies.

Test them separately with test accounts.

Who can change the rules?

- Contract upgrades, pauses and configuration roles
- Account modules and recovery powers
- External contracts and their controllers
- Fixed code still needs a plan for bugs

Work that happens offchain

- Keepers, provers, relayers and sponsored transactions
- Fresh oracle inputs and inherited protocols
- Replacement operators need inputs, permission and funds
- Some failures call for a safe read-only mode

The network underneath the app

- Internet access and reachable peers
- Execution and consensus clients
- Validators, transaction inclusion and finality
- Required history and network lifetime

An L2 recovery path has conditions

OP Stack documents an L1 route during sequencer outages.

Check the chosen chain and account type.
Budget fees and delay.

Forced inclusion and withdrawal are separate tests.

One file and an accepted reference

- Rehearsal walkthrough: planned checks
- File identity and public records
- Expected content reference accepted in advance
- Remaining inputs: RPC, gateway and retained bytes

A different route to the same file

First retrieval

Fetch through IPFS.
Compare against the accepted reference.

Planned check

Replacement retrieval

Disable the selected IPFS path.
Fetch through Arweave.
Check the same reference.

Planned check

Bad bytes and a missing response

INVALID

Retrieved bytes fail the expected check.

Inject a changed local fixture.

Planned check

UNAVAILABLE

The selected source returns no usable response.

That says nothing about every other copy.

Planned check

You accepted release A.
The controller publishes release B.
Both have valid hashes.

Can you keep using A?
What shared state can still change?

The attacker has the right key

One essential task in your app

- Choose the task users cannot afford to lose
- Trace what it needs and who controls it
- Remove the team's services, credentials and help
- Repeat with the team's authority in hostile hands
- Record the recovery path, effort and remaining trust

Recovery has to be usable

- An export needs a working importer
- Copies need people and funding
- Recovery needs affordable time, fees and tools
- The user has to recognize when to switch

Let their work outlive yours

Resources and discussion

- Ethereum File System: efs.eth.limo
- Working Sepolia app: app.efs.eth.limo
- EFS overview video: youtu.be/gZl711IriSM
- Take-home exercise: one task, absence, hostile control
- Sources and deployment limits are in the speaker notes

App and data recovery checks

- Release, source, dependency versions and runtime scripts
- Device, browser, local state and signing interface
- Name controllers, resolver, renewal and accepted reference
- Storage copies, gateways and retention funding
- RPC basis, index definitions, freshness and completeness

Authority and computation checks

- Login, signer transport, recovery and decryption keys
- Proxy, pause, configuration and account-module powers
- Keepers, provers, relayers, bundlers and paymasters
- Oracle freshness, tokens, bridges and inherited upgrades
- Permission, inputs and funding for replacement operators

Network, chain and exit checks

- Internet paths, cloud accounts, peers and bootstrapping
- Execution, consensus, validators, inclusion and finality
- Required history and testnet lifetime
- L2 data, settlement, upgrade powers and recovery routes
- Usable import, continued work, shared operators and cost

Immutable programs and changing tools

Exploratory proposal

Retain a program and versioned execution rules while proving tools evolve.

EIP-8288 is Draft.

Application question

Can a replacement operator still perform every required computation?

Compatibility has a maintenance cost.

What the EFS demonstration establishes

- One accepted reference and one artifact
- Alternate carrier retrieval, mismatch and unavailable-path checks
- Separate authority analysis for authorized malicious changes
- Still to verify: fresh outputs, deployed powers and cold recovery
- Reading one file does not establish every write or discovery path

Questions worth practicing

- What survives if an RPC, gateway or sequencer disappears?
- What can a compromised controller still change?
- How does a newcomer find the accepted reference?
- What did the demonstration actually prove?
- What should a small team improve first?